

SLIP – 24

Section I: Data Structures [15 Marks]

Q1) Accept the array of n integers from user and sort the array in ascending order by using recursive Counting sort algorithm.

Note: counting sort is applied recursively, one digit at a time (non-negative integers).

Answer:

```
#include <stdio.h>

/* one counting-sort pass on the digit given by exp (1, 10, 100 ...) */ void
countingPass(int a[], int n, int exp)
{
    int out[100], count[10] = {0}, i;

    for(i = 0; i < n; i++)
        count[(a[i] / exp) % 10]++;

    for(i = 1; i < 10; i++)
        count[i] += count[i - 1];

    for(i = n - 1; i >= 0; i--)
    {
        out[count[(a[i] / exp) % 10] - 1] = a[i];
        count[(a[i] / exp) % 10]--;
    }

    for(i = 0; i < n; i++)
        a[i] = out[i];
}

/* recursive counting sort: sorts digit by digit */
void countingSort(int a[], int n, int max, int exp)
{
    if(max / exp == 0)
        return;

    countingPass(a, n, exp);
    countingSort(a, n, max, exp * 10);
}

int main()
{
    int a[100], n, i, max;
```

```

    printf("Enter n: ");
    scanf("%d", &n);

    if(n <= 0 || n > 100)
    {
        printf("n must be between 1 and 100");
        return 0;
    }

    printf("Enter %d non-negative integers:\n", n);
    for(i = 0; i < n; i++)        scanf("%d", &a[i]);

    max = a[0];
    for(i = 1; i < n; i++)
    {
        if(a[i] > max)
            max = a[i];
    }

    countingSort(a, n, max, 1);

    printf("Sorted array: ");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}

```

Section II: Database Management Systems [15 Marks]

Q 2) Consider the following database (primary keys are underlined).

Bus (Busno , capacity ,depot_name)

Route (Routeno ,source ,destination ,no_of_stations)

Relationship : Bus-Route : Many to One.

Constraint: Bus capacity should not be null.

Q 2(A)

A. Create the above database in PostGreSQL and insert sufficient records. [5 marks]

Answer:

```

CREATE TABLE Route(
Routeno INT PRIMARY KEY,
source VARCHAR(50),
destination VARCHAR(50),
no_of_stations INT
);

CREATE TABLE Bus(      Busno
INT PRIMARY KEY,      capacity
INT NOT NULL,      depot_name
VARCHAR(50),
      Routeno INT REFERENCES Route(Routeno)
);

INSERT INTO Route VALUES
(41,'Pune station','Baramati',12),
(42,'Pimpri','Hadapsar',2),
(43,'Pimpri','Vimannagar',6);

INSERT INTO Bus VALUES
(201,40,'Swargate',41),
(204,15,'Sangamner',42),
(205,30,'Sangamner',43);

```

Q 2(B)

B. Execute the following queries in PostGreSQL (any 3) [6 marks]

i. Count total buses at depot "Sangamner".

Answer:

```

SELECT COUNT(*) FROM Bus
WHERE depot_name = 'Sangamner';

```

ii. Delete all buses having capacity less than 20.

Answer:

```

DELETE FROM Bus
WHERE capacity < 20;

```

iii. List all buses on route number 41.

Answer:

```

SELECT * FROM Bus
WHERE Routeno = 41;

```

iv. Find the maximum number of stations.

Q 2(C)

C. Write a query to delete all the buses of destination “Baramati”. [4 marks]

Answer:

```
DELETE FROM Bus
WHERE Routeno IN
(
    SELECT Routeno FROM Route
    WHERE destination = 'Baramati'
);
```

Q 3) Viva [5 marks]